

Sprint 7 Platform Progress

Reliable AI workflows, persistent orchestration,
notebook-scoped execution, and full-stack hardening

Capstone project update for the merged Sprint 7 branch • March 2026

Sprint 7 Delivery Snapshot

Merged in GitLab MR !25 on March 5, 2026. Consolidates through-put, scope, and issue coverage for the branch.

292 total commits

Files changed

254

Issues linked

46

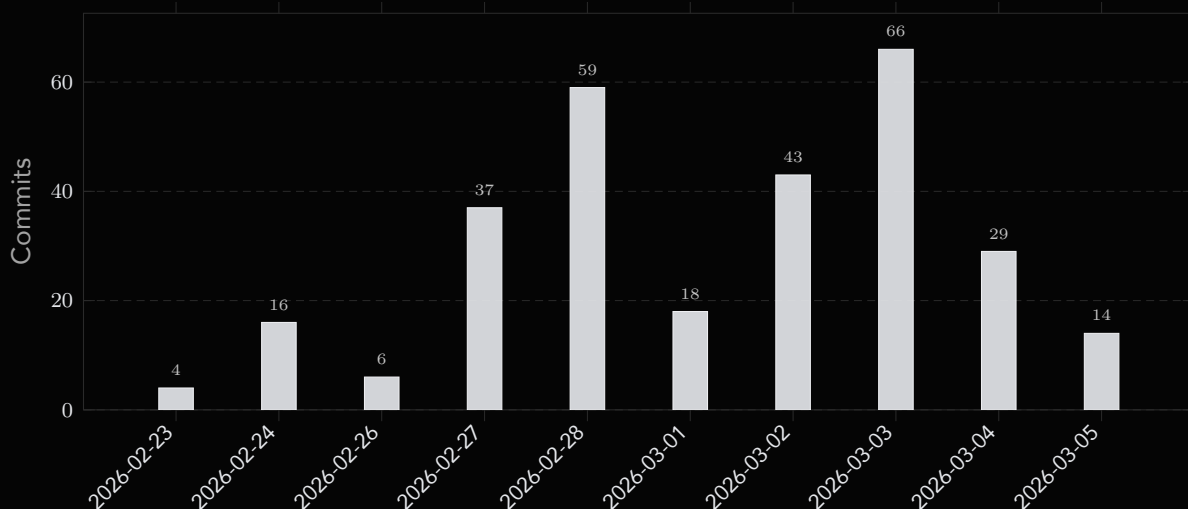
Touched test files

56

Net code delta

**+45,012 /
-7,185**

Commits per Day Across the Sprint Branch



Issue status

36 closes / 10 addresses

Changed file mix

**157 frontend / 79 backend /
17 docs+other**

Interpretation

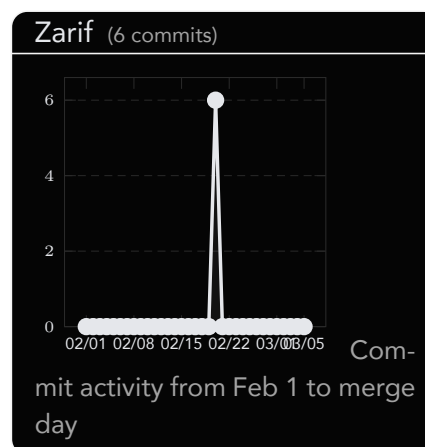
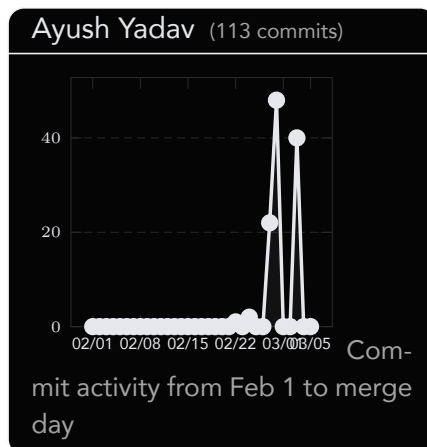
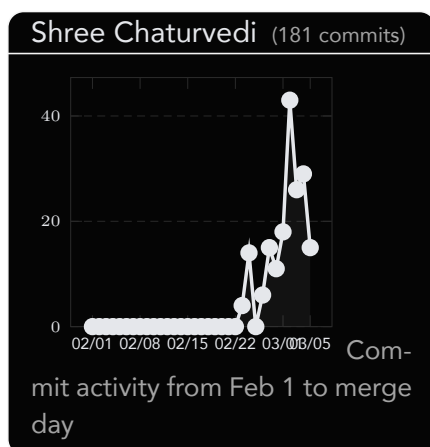
**Architecture-heavy
sprint with sustained
commit density**

Sprint Narrative

Sprint 7 delivered a full-stack architectural jump rather than a collection of isolated fixes. The branch started by restructuring onboarding and planning, then expanded into shared agentic infrastructure, persistent preprocessing orchestration, notebook context threading, and a significantly more inspectable NL-to-SQL experience.

Four themes framed the sprint:

- **NL-to-SQL v2:** replaced a placeholder path with a streamed, explanation-rich, approval-gated workflow.
- **LangGraph preprocessing runtime:** introduced persisted state, step lifecycle control, approval gating, interruption handling, and run rehydration.
- **Multi-notebook + shared shell:** threaded notebookId through APIs, tool calls, and UI state while removing duplicated chat-shell behavior.
- **Refinements and hardening:** virtual scrolling, artifact/image rendering, dataset coercion, SQL validation fixes, auth/profile fixes, and compile/test hardening.



NL-to-SQL v2: Streamed, Reviewable, and Repairable

The most visible user-facing gain in Sprint 7 was the shift from a one-shot NL query path to a streamed workflow that exposes what the system is doing while it works. Instead of hiding model reasoning behind a loading spinner, the UI shows phase progress, model-work blocks, and a final review surface before execution.

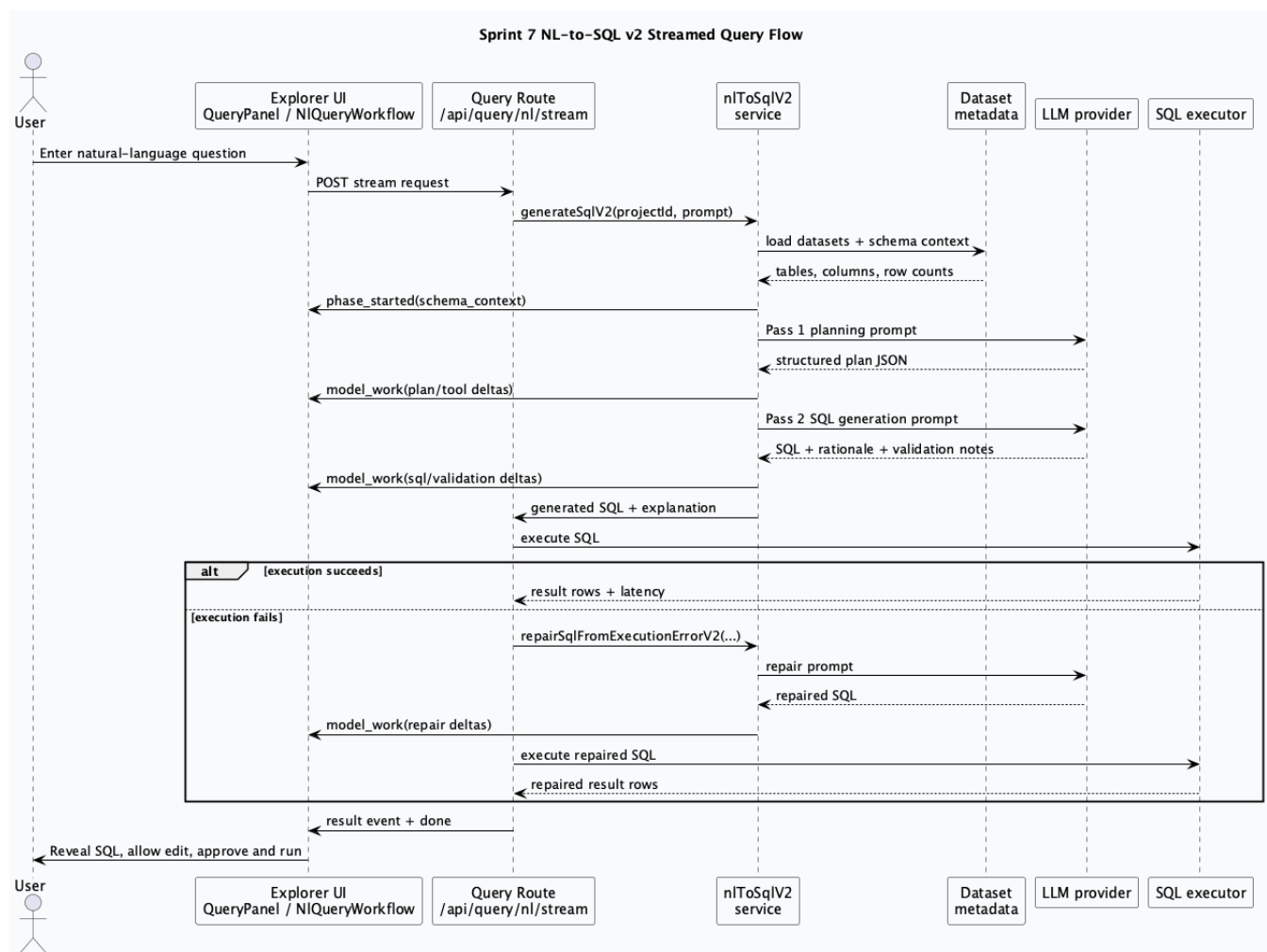


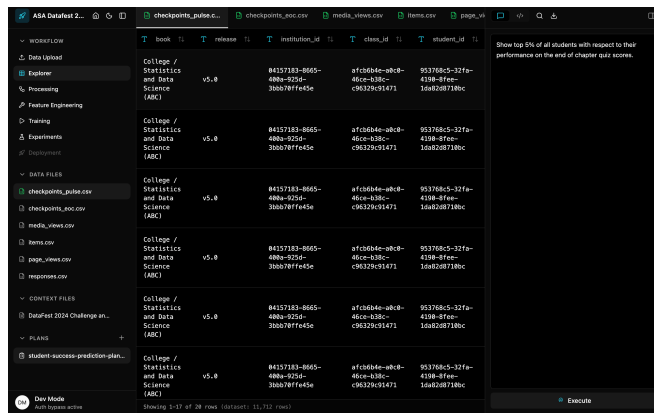
Figure 1: Sequence view of the Sprint 7 NL-to-SQL v2 path. The route streams progress and model-work events, and failed execution can trigger a repair pass instead of ending in a dead stop.

Technical significance

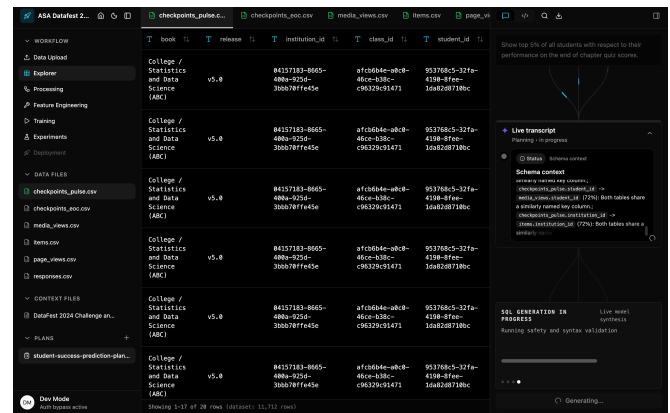
- The backend emits `phase_*` events and separate `model_work_*` events, so the frontend can distinguish workflow status from rich transcript rendering.
- Generation is two-stage: structured planning first, SQL generation second. This allows table selection, joins, assumptions, and validation notes to be surfaced independently of the final SQL.
- Execution is human-in-the-loop. Users can review and edit generated SQL before approving it, which materially reduces the risk of hidden model mistakes.
- Failure handling improved: validation, fallback routing, deterministic fallback behavior, and execution-time repair are all explicit parts of the pipeline.

Preprocessing Runtime: Stateful Orchestration Instead of a Linear Wizard

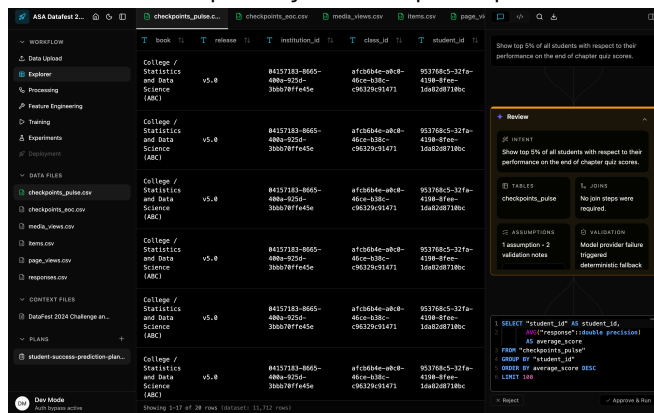
Sprint 7's deepest backend work was the migration of preprocessing to a persisted runtime with explicit step lifecycle rules. The resulting system can track proposals, materialized code, execution, validation, approval, divergence, interruption, and rehydration instead of treating preprocessing as a throwaway



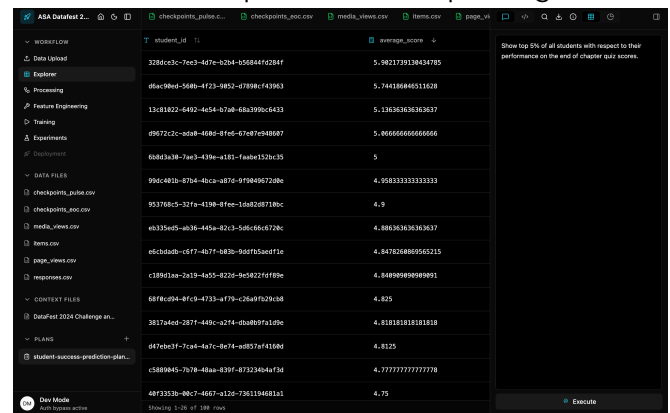
(a) Prompt entry in the Explorer panel



(b) Live transcript and streamed planning state



(c) Structured review and generated SQL



(d) Approved SQL with executed result set

Figure 2: The NL-to-SQL v2 user flow captured from the sprint presentation environment.

chat exchange.

What changed technically

- Run persistence moved into a dedicated repository so preprocessing state survives reloads and can be rehydrated into the UI.
- Cell metadata and notebook bindings became part of the orchestration model, keeping notebook execution as the authoritative execution record while still projecting structured timeline cards in the chat surface.
- Reliability handling was expanded for provider quota failures, stream aborts, replay incompatibilities, and approval-driven branching.
- The runtime now supports checkpointing and divergence detection, which makes longer preprocessing sessions practical instead of fragile.

Multi-Notebook Support, Shared Agentic Shell, and Staged Planning

Sprint 7 also improved product coherence. The frontend now uses a shared agentic shell around a reusable loop instead of separate training and preprocessing chat implementations. Notebook state moved from a single-project assumption to explicit multi-notebook CRUD with notebook-scoped execution.

Output surface	What Sprint 7 added
Explanation metadata	Intent summary, selected tables, join plan, filters, aggregations, assumptions, validation notes, confidence, warning level, and reliability tier
Streaming contract	Ordered phase events plus model-work block start/delta/complete events over the NDJSON stream
Safety layer	Read-only SQL validation, identifier normalization, repair flow after execution failures, and explicit approval before run
UX refinement	SQL reveal block, progress transcript, model-work panel, suggestion chips, and cached-result reuse when approved SQL is unchanged

Table 1: Why the new NL-to-SQL stack is more inspectable than a generic “AI generated SQL” feature.

Architectural impact

- **Reduced duplication:** training and preprocessing now share loop, messaging, tool rendering, and shell layout primitives.
- **Notebook-scoped execution:** notebookId is threaded through routes, tools, and state, which prevents multi-tab workflows from stepping on one another.
- **Clearer onboarding:** the upload → intent → planning → processing → review flow turns project setup into an explicit pipeline instead of a loose set of forms.

Engineering Quality and Refinement

The sprint was also heavy on lower-level refinement. The branch touched 56 test files and resolved issues across the data viewer, notebook runtime, preprocessing reliability, and UI consistency. These items matter because they keep the new architectural work usable under real project conditions.

Problem	Sprint 7 fix	Impact
Large table views became inefficient	Replaced the older table strategy with virtual scrolling and sticky-header improvements	Explorer can handle larger result sets with less UI lag and cleaner viewport behavior
Notebook output support was too narrow	Added execution artifact handling and image rendering for outputs such as Matplotlib figures	Notebook cells can now show richer runtime artifacts without manual workarounds
Dataset ingestion was fragile	Hardened value coercion, malformed input handling, and upload failure paths	Fewer silent ingest failures and better resilience on real datasets
SQL and build regressions slowed iteration	Fixed validator edge cases, compile blockers, and frontend TS regressions	Sprint features stayed shippable while architectural work was landing
Sidebar / PDF / chooser UX had polish gaps	Addressed animation, centering, toolbar sizing, chooser behavior, and focus-state inconsistencies	The application feels more production-grade during demo and day-to-day use

Table 2: Representative refinement work that made the headline features sustainable.

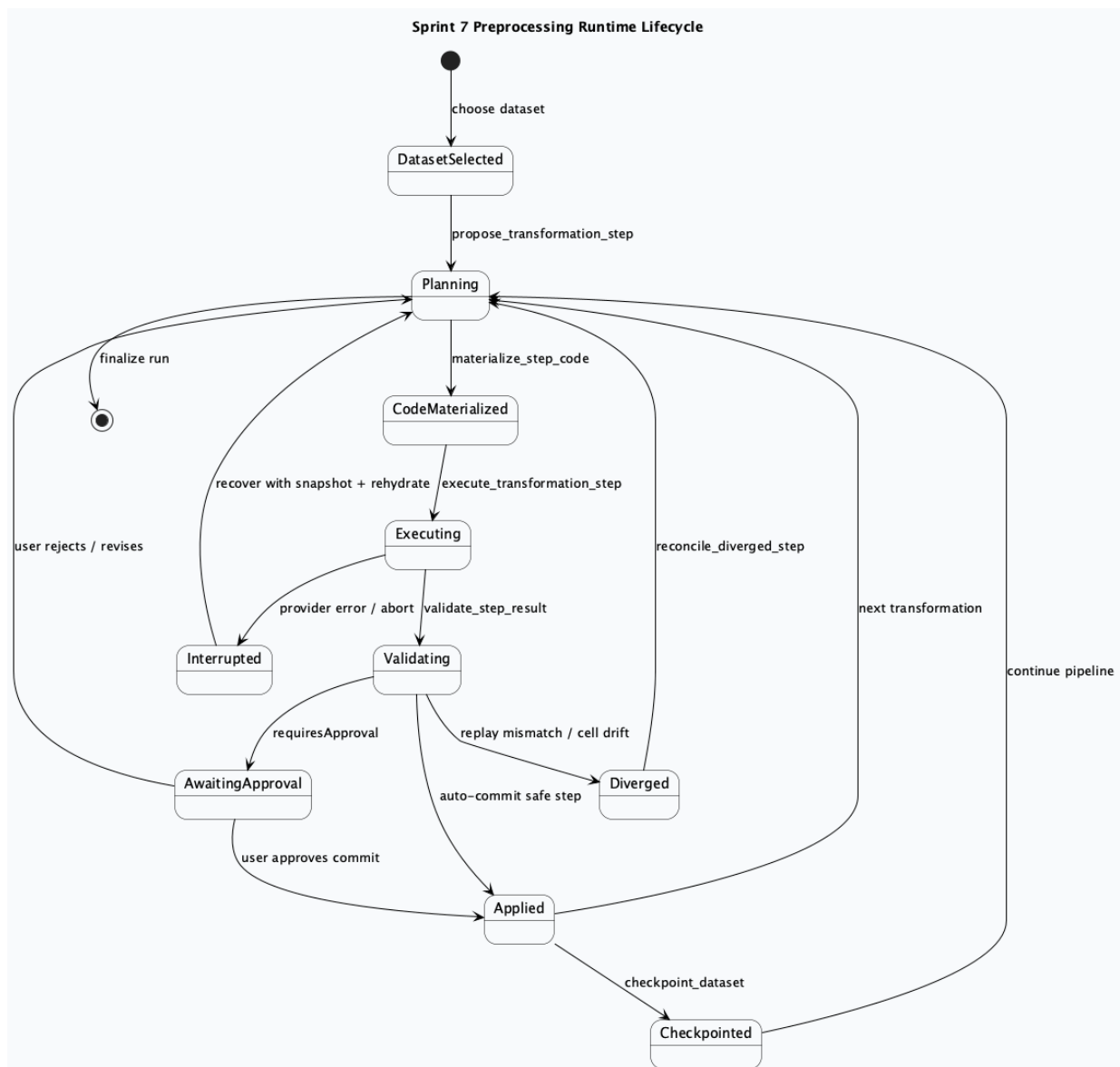


Figure 3: State view of the preprocessing lifecycle. Approval, divergence, interruption, and checkpoint flows are first-class runtime concerns rather than ad hoc UI state.

Verification-oriented evidence

- **Touched test files:** 56
- **Frontend files changed:** 157
- **Backend files changed:** 79
- **Docs / other files changed:** 17

What Sprint 7 Unlocked

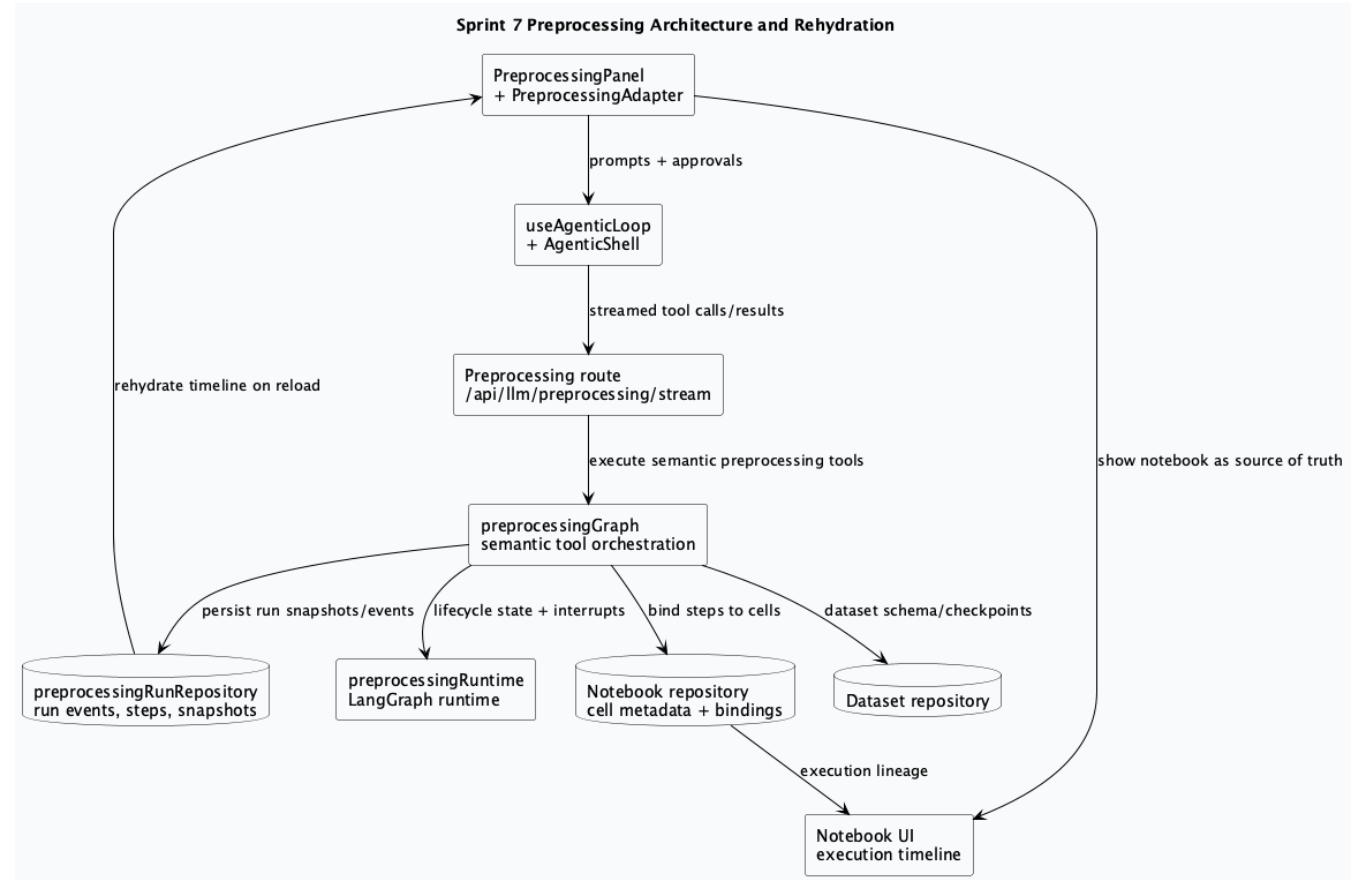


Figure 4: Architecture view of Sprint 7 preprocessing. Run state, notebook cell bindings, and dataset context are persisted so the UI can recover and continue safely after refreshes or interruptions.

- **More inspectable AI workflows.** Users can see planning and execution logic instead of trusting opaque text generation.
- **More reliable preprocessing execution.** Run persistence, approval gating, divergence handling, and checkpointing make preprocessing robust enough for longer workflows.
- **Better notebook and project organization.** Multi-notebook support and notebook-scoped tooling make concurrent workflows practical.
- **A stronger platform foundation.** Shared shell infrastructure and staged onboarding reduce future feature cost and make the product easier to extend.

Appendix: Issue Coverage Evidence

The following table maps each issue explicitly linked in Sprint 7 MR !25 to its resolution status and relevant commits.

Group	Issue	Status	Title
Critical & High — Sprint 7 Core	#28	Closes	Critical: Notebook Scroll Lock When Hovering Code Cells
Critical & High — Sprint 7 Core	#27	Closes	Critical: Feature Engineering Page Crashes on Navigation

Group	Issue	Status	Title
Critical & High — Sprint 7 Core	#42	Closes	High: No Multiple Notebooks Support
Critical & High — Sprint 7 Core	#41	Closes	High: Processing Tab UI Misalignment and Improvements
Critical & High — Sprint 7 Core	#39	Closes	High: SQL Editor Intellisense Incomplete
Critical & High — Sprint 7 Core	#38	Closes	High: Plan Selector Hover Blocks UI Elements
Critical & High — Sprint 7 Core	#36	Closes	High: Data Upload Top Bar Should Only Show Description Field
Critical & High — Sprint 7 Core	#33	Closes	High: Chat File Upload UX/Functionality
Critical & High — Sprint 7 Core	#25	Closes	Dataset upload hardening: large-file 413 handling + resilient timestamp coercion
Critical & High — Sprint 7 Core	#44	Closes	Medium: Feature Engineering Page Needs Implementation
LangGraph Runtime Migration (SEQ series)	#48	Closes	[META] LangGraph runtime migration umbrella
LangGraph Runtime Migration (SEQ series)	#50	Closes	[META] Preprocessing run rehydration umbrella
LangGraph Runtime Migration (SEQ series)	#51	Closes	[META] Step-cell metadata and divergence reconciliation umbrella
LangGraph Runtime Migration (SEQ series)	#45	Closes	[SEQ-01] Decide Processing state model
LangGraph Runtime Migration (SEQ series)	#52	Closes	[SEQ-02] Add LangGraph runtime foundation and preprocessing state schema
LangGraph Runtime Migration (SEQ series)	#53	Closes	[SEQ-03] Implement preprocessing lifecycle graph nodes/transitions with interrupts
LangGraph Runtime Migration (SEQ series)	#54	Closes	[SEQ-04] Wire LangGraph execution path and add parity regression tests
LangGraph Runtime Migration (SEQ series)	#55	Closes	[SEQ-05] Persist canonical step-cell metadata in notebook and run events
LangGraph Runtime Migration (SEQ series)	#56	Closes	[SEQ-06] Implement backend divergence detection and reconciliation flow
LangGraph Runtime Migration (SEQ series)	#46	Closes	[SEQ-07] Persist preprocessing approval actions as backend orchestration decisions
LangGraph Runtime Migration (SEQ series)	#57	Closes	[SEQ-08] Add backend preprocessing run snapshot APIs for rehydration
LangGraph Runtime Migration (SEQ series)	#58	Closes	[SEQ-09] Hydrate preprocessing timeline from backend snapshots on reload/tab restore
LangGraph Runtime Migration (SEQ series)	#47	Closes	[SEQ-10] Make replay compatibility backend-authoritative in UI
LangGraph Runtime Migration (SEQ series)	#49	Closes	[SEQ-11] Deprecate/cleanup legacy preprocessing analyze-refine-execute endpoints
LangGraph Runtime Migration (SEQ series)	#59	Closes	[SEQ-12] Fix Processing tab history persistence across refresh/reset
LangGraph Runtime Migration (SEQ series)	#60	Closes	[SEQ-13] Harden preprocessing runId ownership and backend-only run creation
LangGraph Runtime Migration (SEQ series)	#62	Closes	[LangGraph][Extreme] Preprocessing reliability under Gemini quota failures
Explorer, SQL & Data Fixes	#22	Closes	SQL UX does not handle identifiers with spaces (unquoted column names)
Explorer, SQL & Data Fixes	#21	Closes	Explorer Query Builder editor turns black after phase navigation in light mode

Group	Issue	Status	Title
Explorer, SQL & Data Fixes	#20	Closes	Clicking Data Files while Explorer is locked results in silent redirect/no-op
Explorer, SQL & Data Fixes	#24	Closes	Feature Engineering Generate AI plan can complete with no visible output
Explorer, SQL & Data Fixes	#76	Addresses	Implement virtual scrolling for database table views
Explorer, SQL & Data Fixes	#84	Addresses	Reduce query execution table font size to match UI sizing
Explorer, SQL & Data Fixes	#85	Addresses	Fix table header scrolling out of view during vertical scroll in explorer tab
Explorer, SQL & Data Fixes	#102	Addresses	Improve dataset chooser dialog UI/UX and enforce selection
Build, Compile & Regression Fixes	#92	Closes	Frontend build hardening: fix TS compile blockers in data/preprocessing UI
Build, Compile & Regression Fixes	#94	Closes	Processing tabs must have isolated notebook state (create/reset/delete lifecycle)
Build, Compile & Regression Fixes	#95	Closes	Frontend build regression: TS2352 in NIFlowConnector test blocks build
Notebook & Cell Improvements	#65	Addresses	Notebook cell improvements: execution state, UI styling, and markdown collapsing
UI/UX Fixes	#83	Addresses	Add visual background effect to home page
UI/UX Fixes	#96	Addresses	Reduce PDF viewer toolbar height to match DataTable column header
UI/UX Fixes	#97	Addresses	Center AutoML logo in collapsed sidebar on home screen
UI/UX Fixes	#98	Addresses	Sidebar project icons jump 2-5px when collapsing sidebar
UI/UX Fixes	#99	Addresses	Query panel collapse animation has visual glitch when PDF is open
Sprint 6 Leftovers	#18	Closes	Tool calls fail when Thinking mode is OFF in Training chat
Sprint 6 Leftovers	#19	Closes	File upload shows failure state until page refresh (false negative)

Appendix: Commit Timeline Highlights

These milestone commits trace the branch across onboarding, shell unification, preprocessing, notebooks, and NL-to-SQL v2.

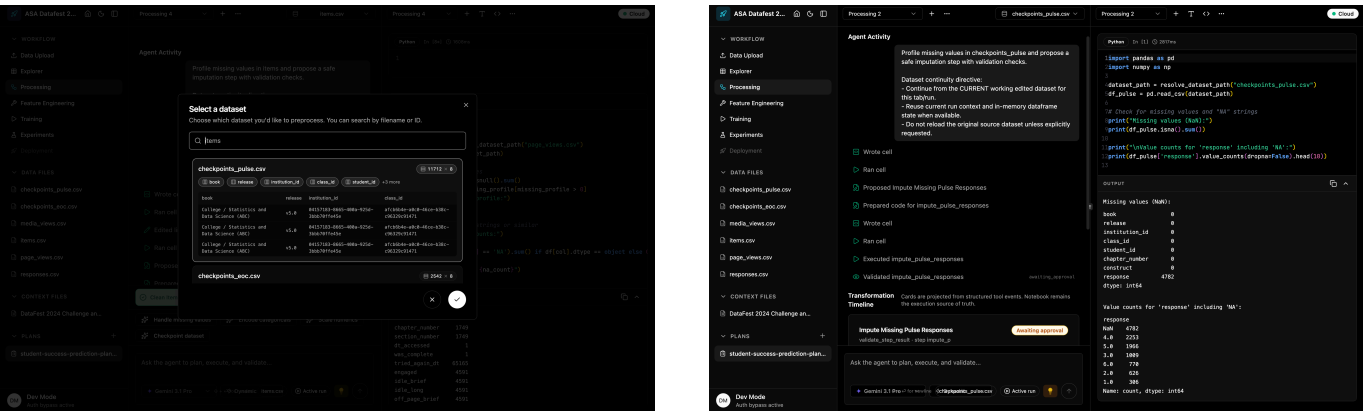


Figure 5: UI evidence of the new preprocessing runtime.

Phase highlight	Commit	Date	Evidence
Staged onboarding work-flow	abcc0ce	2026-02-23	feat(frontend): add staged upload-to-planning workflow
Shared agentic shell	e12fa5f	2026-02-26	feat(frontend): unify training and preprocessing on shared agentic shell
Multi-notebook backend	1e94b2b	2026-02-27	feat(notebooks): support list/create/re-name/delete notebook service flows for issue #42
LangGraph semantic run-time	c54196b	2026-02-28	feat(langgraph): implement preprocessing tool plumbing and structured outputs
Persistent preprocessing replay	700418f	2026-02-26	feat(preprocessing): persist semantic graph runs and replay checks
NL-to-SQL v2 backend	bb3f82c	2026-03-03	backend: add nl2sql v2 service and dedicated config knobs
Streamed NL workflow UI	009c7b7	2026-03-04	feat(frontend): refactor NIQueryWorkflow for streaming pipeline events

Appendix: File Change Summary

Area	Changed files	Insertions	Deletions
Backend	79	15790	639
Frontend	157	28109	5717
Docs / Other	17	1113	829
Touched test files	56	—	—

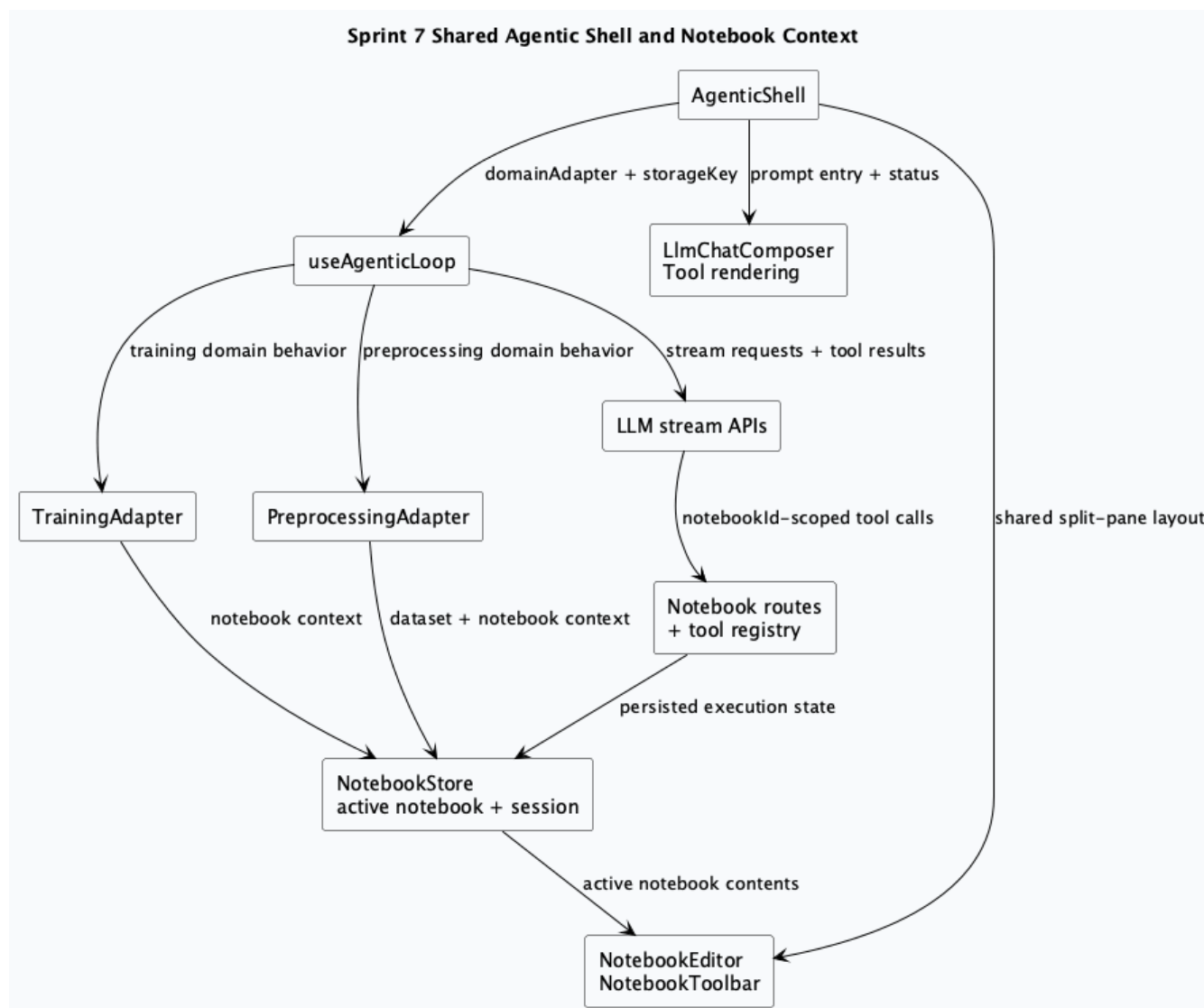


Figure 6: Shared shell architecture. A common `AgenticShell` and `useAgenticLoop` coordinate domain adapters while notebook context is threaded into tool execution.

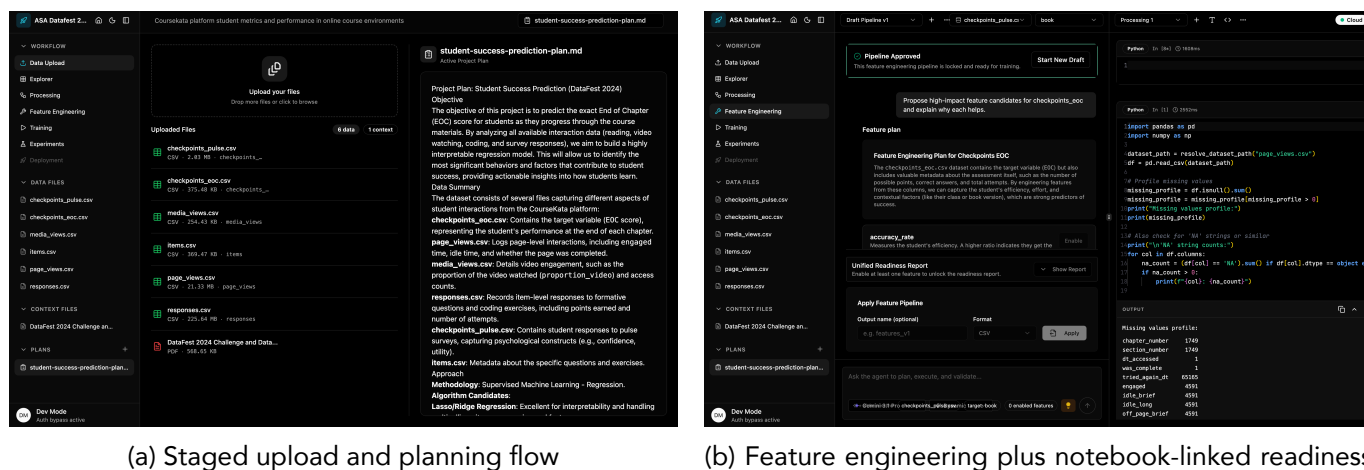


Figure 7: The shared shell and notebook work unified onboarding, preprocessing, feature engineering, and notebook execution into a cohesive set of surfaces.